

# PA1 Written Questions

Munahid Iqbal

CMSC 471 Section 01, Fall 2026

## Question 1: Heuristic design and admissibility

All three of my heuristics come from one main idea, the only thing that changes is the numbers, and those depend on whether the cost is steps, scrabble, or frequency. I compare my word to the goal word letter by letter and find every spot where the letters don't match. Every one of those spots has to get changed at some point, since there's no way to end up at the goal word while one of them is still wrong. For each wrong spot, I look at the very last action that changes it. That last change has to put the goal letter in, because once a spot gets changed for the last time it can't ever change again, and it has to be correct by the end. One action only ever changes one spot, so the last changes of two different spots are always two different actions. Since the solution has to pay for every single one of those last changes no matter what path it takes, I can add up the cheapest each one could possibly cost, and that total is my guess.

### Steps

1. **Give a precise formula or pseudocode:**

$h(s)$  = the number of spots where  $s$  has a different letter than the goal

2. **Explain what unavoidable remaining cost it lower-bounds:** every wrong spot needs at least one

action to fix it, because the letter there is wrong and it can't fix itself. One action only changes one spot. And with steps, every single action costs exactly 1 no matter what letter it is. So a solution can never cost less than (number of wrong spots) \* 1.

3. **Explain why it cannot overestimate the true optimal remaining cost:** my count is made up only of changes the solution is forced to make, so I never add in a "maybe" cost. If anything I leave stuff out, like the times a spot gets changed more than once on its way to the right letter. Leaving stuff out can make my guess too low, but it can never make it too high, and being too low doesn't break anything. Also, the goal word has no wrong spots, so  $h = 0$  there.

## Scrabble

1. **Give a precise formula or pseudocode:**  $h(s) = \text{add up } \text{scrabble\_value}[\text{goal}[i]] \text{ for every spot } i \text{ where } s[i] \text{ is different from } \text{goal}[i]$
2. **Explain what unavoidable remaining cost it lower-bounds:** going back to the main idea, the last change to a wrong spot is forced to put the goal letter in. With scrabble costs, what an action costs is just the value of the letter you're putting in. So the last change to wrong spot  $i$  has to cost exactly  $\text{scrabble\_value}[\text{goal}[i]]$ , no matter which solution you end up taking. Every wrong spot's last change is its own separate action, so the solution pays for every single one of them.
3. **Explain why it cannot overestimate the true optimal remaining cost:** I don't have to guess anything about which actions the solution picks. Whichever way it goes, it has to include one last change per wrong spot, and each one of those costs exactly what its goal letter is worth. Since those actions have to be in the solution, their prices have to be in the total, so my sum can never come out bigger than the real cost. The goal has no wrong spots so  $h = 0$  there, and it's not the lazy always-return-0 kind either (for why  $\rightarrow$  ask,  $h(\text{why}) = 1 + 1 + 5 = 7$ ).

## Frequency

Before any searching happens, my dc.py builds a table called MIN\_RARITY. For every word length, spot, and letter, it stores the smallest rarity of any dictionary word that has that letter in that spot. It gets filled in with one loop over the whole dictionary when the program starts.

1. **Give a precise formula or pseudocode:**

$h(s) = \text{add up } (1 + \text{MIN\_RARITY}[(\text{len}(s), i, \text{goal}[i])])$  for every spot  $i$  where  $s[i]$  is different from  $\text{goal}[i]$

2. **Explain what unavoidable remaining cost it lower-bounds:** going back to the main idea again, the last change to a wrong spot makes some real word  $w$ , and that word has to have the goal letter in that spot (same idea as scrabble). With frequency costs, that action costs  $1 + \text{rarity}(w)$ . However, I have no idea which word  $w$  the solution will pick. But I don't need to know it, because whatever  $w$  turns out to be, its rarity can't be smaller than the rarity of the cheapest dictionary word that has the right letter in the right spot. That cheapest-possible rarity is exactly what my MIN\_RARITY table stores, so the last change to spot  $i$  has to cost at least  $1 + \text{MIN\_RARITY}[(\text{len}(s), i, \text{goal}[i])]$ .
3. **Explain why it cannot overestimate the true optimal remaining cost:** every number I add is less than or equal to the cost of an action that has to happen, and each spot's last change is a different action, so my total can never come out above the real cost. This guess is honestly way too low a lot of the time, for why -> ask,  $h(\text{why}) = 10.19$  but the real cost is 38.86. That's because MIN\_RARITY grabs the single cheapest word in the whole dictionary for each spot, and real solutions can't do that, they have to travel through normal everyday words whose rarities are way bigger. Being too low doesn't break anything though, it just means  $A^*$  gets less help out of the guesst.

## Question 2: Does the heuristic actually help?

I used the search functions already in search.py, called like `uniform_cost_search(problem, display=True)` and `astar_search(problem, display=True)`, and I made a brand new DC problem for every single run. For the node counts, I just used the number that the function itself prints out.

| Word pair      | Cost measure | UCS cost | UCS nodes expanded | A* cost | A* nodes expanded | Reduction |
|----------------|--------------|----------|--------------------|---------|-------------------|-----------|
| WHY →<br>ASK   | Steps        | 5.00     | 773                | 5.00    | 16                | 97.9%     |
| WHY →<br>ASK   | Scrabble     | 9.00     | 765                | 9.00    | 13                | 98.3%     |
| WHY →<br>ASK   | Frequency    | 38.86    | 808                | 38.86   | 510               | 36.9%     |
| BEAR →<br>DUCK | Steps        | 4.00     | 691                | 4.00    | 6                 | 99.1%     |
| BEAR →<br>DUCK | Scrabble     | 11.00    | 2981               | 11.00   | 6                 | 99.8%     |
| BEAR →<br>DUCK | Frequency    | 38.24    | 1084               | 38.24   | 319               | 70.6%     |

Reduction =  $100 * (\text{UCS nodes} - \text{A* nodes}) / \text{UCS nodes}$ .

1. **Did UCS and A\* always return the same optimal cost? If not, identify and investigate the**

**discrepancy:** yes, all six runs matched exactly (5.00, 9.00, 38.86, 4.00, 11.00, 38.24 each way).

Both searches find the best possible answer, and my heuristic never guesses too high, so A\* has no way to end up with a worse cost than UCS. They don't always take the same path to get there, but the final cost always comes out the same.

2. **For each word pair, under which cost measure did A\* produce the largest percentage**

**reduction in expanded nodes?:** Scrabble for both word pairs.

3. **For each word pair, under which cost measure was the heuristic least informative?:**

Frequency for both word pairs.

4. **Explain the differences using the definitions of your heuristics, not runtime alones:** All three of my heuristics add up a "cheapest possible last change" for each wrong spot, so the real question is which cost measure makes that cheapest-possible number closest to what a real solution actually pays.

**Steps:** my h here is just the count of wrong spots, and for these two problems there are paths that fix the word without ever wasting a move, so the guess is basically the true answer. That's why A\* almost walks straight to the goal (16 and 6 nodes), while UCS has no guess at all and just expands basically every word that costs less than the final answer (691–773 nodes).

**Scrabble:** the last change into each wrong spot really does cost exactly what the goal letter is worth, so my guess can be dead on. For bear -> duck,  $h(\text{bear}) = 2 + 1 + 3 + 5 = 11$ , which is exactly the true remaining cost, so every word on the best path looks perfect to A\* and does it (only 6

nodes). UCS doesn't get any of that info, so it floods the search with words that are cheap to enter (vowels, T, S, R, N all cost 1) and ends up expanding 2,981 of them before it finally pops duck.

**Frequency:** this one is the loosest guess, and the reason is baked into the heuristic. MIN\_RARITY grabs the cheapest word in the entire dictionary for each spot/letter, but real solutions have to travel through normal words with normal (bigger) rarities. So the guess ends up tiny compared to the truth,  $h(\text{why}) = 10.19$  when the real cost is 38.86, only about 26% of it. When the guess is that small,  $f = g + h$  acts basically like just  $g$ , so  $A^*$  explores almost as much as UCS did (510 and 319 instead of 808 and 1084). Still better, just not by a ton.

## Question 3: Reversibility and direction-dependent costs

If I can change kit into kip by switching the last letter, I can change kip back into kit the same way. So all the moves are reversible. But that doesn't automatically mean the move costs the same to undo. Whether it does depends on what the cost is based on, and that's different for each of the three cost measures

**Steps:** Yes. With steps, every action costs 1 no matter which direction it goes. So if I take any path from word A to word B and walk it backwards, I get a path from B back to A with the exact same number of moves and the exact same cost. Since the cheapest path forwards turns into a cheapest-ish path backwards, the two best costs can never be different. Example: dog  $\rightarrow$  cog  $\rightarrow$  cot  $\rightarrow$  cat is 3 steps, and cat  $\rightarrow$  cot  $\rightarrow$  cog  $\rightarrow$  dog is also 3 steps.

**Scrabble:** No. With scrabble, the cost of a move is the value of the letter you're putting in. But when you undo a move, you're putting in a different letter than you did the first time. So the same change can cost

two different amounts depending on direction. An example would be changing the t in kit into a p costs 3, but changing it back costs 1. So, a counterexample is:

kit -> zap costs 14: kit -> kip -> zip -> zap ( $p = 3, z = 10, a = 1$ )

zap -> kit costs 7: zap -> zip -> kip -> kit ( $i = 1, k = 5, t = 1$ )

It's literally the same path walked backwards, and it costs 14 one way and 7 the other way. So no, the best cost does not have to be the same in both directions.

**Frequency:** No. With frequency, the cost of a move is  $1 + \text{rarity of the word you land on}$ . Undoing a move means you land on the other word, and two different words almost never have the same rarity. For whole problems, not just single moves:

why -> ask costs 38.86

ask -> why costs 37.76

So the best cost from A to B does not have to equal the best cost from B to A.

## Question 4: LLM heuristic audit

### 1. Model, exact prompt, and proposed heuristic

- **Model used:** Gemini Thinking
- **Exact prompt:**

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly

one letter. Under the frequency objective, entering word  $w$  costs  $1 + \text{rarity}(w)$ , where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic  $h_0(n)$  is the number of positions in which state  $n$  differs from the goal.

Propose a rarity-aware heuristic intended to be more informative than  $h_0$  while remaining admissible. Define it precisely and justify your proposal.

By "more informative," we mean that the proposed heuristic should be at least as large as  $h_0$  for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

**The proposed heuristic:** the model proposed a heuristic that takes the baseline  $h_0$ , which is the count of wrong spots, and adds the rarity of the goal word on top, but only when you're not already at the goal:

$$\begin{aligned} h_1(s) &= 0 && \text{if } s \text{ is the goal} \\ h_1(s) &= h_0(s) + \text{rarity}(\text{goal word}) && \text{otherwise} \end{aligned}$$

The model's justification was that any path from a non-goal word has to enter the goal word at the end, and entering it costs exactly  $1 + \text{rarity}(\text{goal})$ . Every other word along the way costs at least 1 to enter because costs are  $1 + \text{rarity}$  and rarity can't be negative. The path also needs at least  $h_0(s)$  moves since each move fixes one spot. So the total has to be at least  $(\text{number of moves}) + \text{rarity}(\text{goal}) = h_1(s)$ .

### 3. Determine whether the heuristic is admissible. Give either a general proof or a concrete

**counterexample:** Here is the general proof:

"rarity can't be negative" I checked the actual dictionary and the smallest rarity is 0.016 and none

are negative. So this is true.

"any solution enters the goal word and pays  $1 + \text{rarity}(\text{goal})$ " This is true as long as  $s$  isn't already the goal, which is the only case the formula adds the rarity for.

"every other entered word costs at least 1" This is true since each cost is  $1 + \text{rarity}$  and  $\text{rarity} \geq 0$ .

"at least  $h_0(s)$  moves" This is true, each move changes one spot.

Putting those together, the real cost  $\geq h_0(s) + \text{rarity}(\text{goal}) = h_1(s)$ , so it never guesses too high. It also passes the "more informative" bar,  $h_1 \geq h_0$  everywhere, and strictly bigger at every non-goal state since  $\text{rarity}(\text{ask}) = 4.23$  and  $\text{rarity}(\text{duck}) = 7.23$  are both positive. And  $h_1(\text{goal}) = 0$ .

**Verdict:** the proposal is valid. It's one constant added on top of  $h_0$ , but the constant is genuinely unavoidable cost, so the proof works. Since it's valid, no need for repair.

## 5. Test the verified or repaired heuristic against $h_0$ on the two frequency-cost problems from

**Question 2. Compare solution costs and numbers of expanded nodes, then conclude whether the evidence supports using the proposed heuristic:**

| Pair                  | Algorithm     | Cost  | Nodes expanded |
|-----------------------|---------------|-------|----------------|
| WHY $\rightarrow$ ASK | UCS           | 38.86 | 808            |
| WHY $\rightarrow$ ASK | A* with $h_0$ | 38.86 | 755            |
| WHY $\rightarrow$ ASK | A* with $h_1$ | 38.86 | 634            |

| Pair        | Algorithm  | Cost  | Nodes expanded |
|-------------|------------|-------|----------------|
| BEAR → DUCK | UCS        | 38.24 | 1084           |
| BEAR → DUCK | A* with h0 | 38.24 | 720            |
| BEAR → DUCK | A* with h1 | 38.24 | 302            |

All three searches found the same optimal costs (38.86 and 38.24), which they have to since h1 never guesses too high. But A\* with h1 expanded fewer nodes than with h0 on why -> ask (634 vs 755) and fewer on bear -> duck (302 vs 720).

**Conclusion:** yes, the evidence supports using h1 over h0. Same answers and it is way less work.

- I used online tools and resources to help complete the homework